

Revisão do SOLID

1. Uma empresa de desenvolvimento de software está criando um módulo para cálculo de impostos. O sistema deve ser capaz de calcular diferentes tipos de impostos (ICMS, IPI, ISS) de forma flexível, permitindo a adição de novos impostos no futuro com mínima alteração no código existente.

Para isso, a equipe implementou uma hierarquia de classes onde a classe base **Imposto** que define a interface comum, e as classes derivadas que implementam os cálculos específicos de cada tributo.

A seguir são apresentados os impostos:

O ICMS que cobra 7% em compras fora do estado e 18% dentro do estado;

O IPI que cobra 15% para produtos industrializados;

O ISS que cobra 4 a 5% de impostos municipais.

Você deve projetar as classes definidas acima e testar o polimorfismo.

2. O estagiário desenvolveu um sistema de notificações com forte acoplamento entre a *main* e os mecanismos de envio de notificações, conforme pode ser visualizado abaixo.

```
class SistemaNotificacoes {
    private EmailService emailService;
    private SMSService smsService;

    public SistemaNotificacoes() {
        this.emailService = new EmailService();
        this.smsService = new SMSService();
    }

    public void notificarCliente(String mensagem, String tipo)
    {
        if (tipo.equals("email")) {
            emailService.enviarEmail(mensagem);
        } else if (tipo.equals("sms")) {
            smsService.enviarSMS(mensagem);
        }
    }
}
```

```
}

class EmailService {
    public void enviarEmail(String mensagem) {
        System.out.println("Enviando email: " + mensagem);
    }
}

class SMSService {
    public void enviarSMS(String mensagem) {
        System.out.println("Enviando SMS: " + mensagem);
    }
}
```

O sistema deve suportar notificações via WhatsApp, bem como outros canais de comunicação. Cada nova adição, de um canal, exigirá que classes sejam refatoradas. Aplique o Princípio do Aberto/Fechado e do DIP nos programas acima.

3. A equipe de desenvolvimento projetou uma interface para dispositivos de escritório em um sistema de gerenciamento de equipamentos, mas essa interface está causando problemas a cada inclusão de um dispositivo. Aplique o Princípio da Segregação de Interface

```
interface DispositivoDeEscritorio {
    void imprimir(String documento);
    void digitalizar(String documento);
    void fax(String documento);
    void fotocopiar(String documento);
    void enviarEmail(String documento, String destino);
    void encadernar(String documento);
}

class ImpressoraSimples implements DispositivoDeEscritorio {
    @Override
    public void imprimir(String documento) {
        System.out.println("Imprimindo: " + documento);
    }

    @Override
    public void digitalizar(String documento) {
        throw new UnsupportedOperationException("Esta impressora não digitaliza");
    }
}
```

```
}

@Override
public void fax(String documento) {
    throw new UnsupportedOperationException("Esta impressora não envia fax");
}

@Override
public void fotocopiar(String documento) {
    throw new UnsupportedOperationException("Esta impressora não fotocopia");
}

@Override
public void enviarEmail(String documento, String destino) {
    throw new UnsupportedOperationException("Esta impressora não envia email");
}

@Override
public void encadernar(String documento) {
    throw new UnsupportedOperationException("Esta impressora não encaderna");
}
}

class ScannerProfissional implements DispositivoDeEscritorio {
    @Override
    public void imprimir(String documento) {
        throw new UnsupportedOperationException("Este scanner não imprime");
    }

    @Override
    public void digitalizar(String documento) {
        System.out.println("Digitalizando em alta resolução: " + documento);
    }

    @Override
    public void fax(String documento) {
        throw new UnsupportedOperationException("Este scanner não envia fax");
    }

    // ... outros métodos similares com UnsupportedOperationException
}

class AssistenteDigital {
    public void processarDocumento(DispositivoDeEscritorio dispositivo, String doc) {
        try {
            dispositivo.digitalizar(doc);
        }
    }
}
```

```
        dispositivo.imprimir(doc);
    } catch (UnsupportedOperationException e) {
        System.out.println("Erro: " + e.getMessage());
    }
}
}
```